

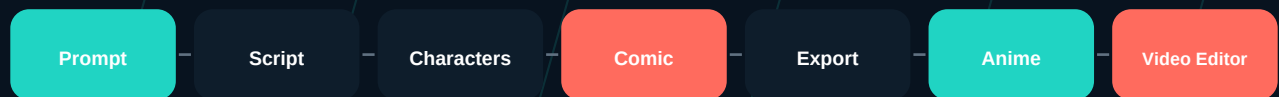


ORIGIN STUDIOS

PRODUCT DIRECTION / IMPLEMENTATION PLAN

Origin Frame: Prompt-to-Comic-to-Anime

A focused production system for turning one story idea into a structured script, a professionally lettered comic or manga, and an editable animated episode.



Prepared for Origin Studios
Generated 17 July 2026
Version 1.0



Contents

This document preserves the approved product direction, architecture, implementation sequence, testing requirements, rollout measures, and operating assumptions.

Summary	4
Product boundaries	4
Product Experience	4
1. Origin Frame dashboard	4
2. Project navigation	5
3. Idea and outline generation	5
4. Script generation and editing	6
5. Characters and visual style	7
6. Comic and manga creation	7
7. Professional lettering	8
8. Story exports	8
One-Click Anime Generation	9
Anime preflight	9
Anime production graph	10
Anime review	10
Video Editor Direction	10
Story integration	10
TimelineDocument version 2	11
Professional media preparation	11
Editor capabilities to complete	12
Data Model Changes	12
Story tables	12
Important contracts	13
API Changes	13
Provider and Job Architecture	14
Comic Rendering	15
Migration and Compatibility	15
Implementation Sequence	15

Phase 1 - Product pivot and contracts

Phase 2 - Prompt, outline, and script

Phase 3 - Character consistency and comic generation

Phase 4 - Publication export

Phase 5 - Anime generation

Phase 6 - Video editor hardening

Phase 7 - Collaboration and production readiness

Test and Acceptance Plan

Unit tests

API integration tests

Provider tests

End-to-end acceptance scenario

Quality gates

Rollout and Success Measures

Assumptions and Defaults

15

16

16

16

16

16

16

17

17

17

17

18

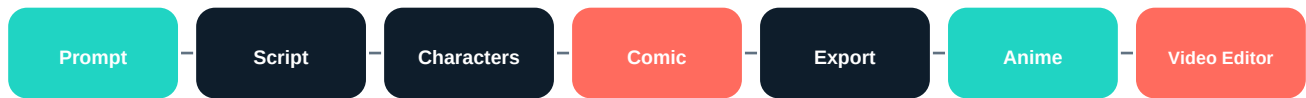
18

18

19

Summary

Origin Frame will pivot from a collection of separate shot-design tools into one focused creative workflow:



The core promise is: "Describe your story, turn it into a polished comic, then convert it into an editable animated episode."

The existing authentication, workspaces, projects, Supabase Storage, immutable versions, generation jobs, collaboration infrastructure, cost tracking, and video editor will be retained. The current seven Shot Design modes and generic scene-first interface will be retired from the primary UI.

Product boundaries

- Optimize the first release for 8-24-page short stories.
- Support left-to-right comics and right-to-left manga.
- Use guided stages with approval before expensive generation.
- Allow "Generate all" inside an approved stage.
- Generate artwork without baked-in dialogue.
- Render dialogue, captions, narration, and sound effects as editable lettering.
- Export comics as PDF, CBZ, and page-image ZIP.
- Generate a 1-3-minute anime draft with motion, voices, captions, and an editable timeline.
- Defer final cloud MP4 export until the editor and generation handoff are stable.
- Remove or hide UI controls that are not genuinely functional.

Product Experience

1. Origin Frame dashboard

Keep `/studio/frame` as the main dashboard, but divide it into:

- Story Projects
- Video Edits
- Recent Exports

Each story card shows:

- Cover or latest page
- Title
- Comic or manga format
- Current stage
- Page count
- Generation progress

- Last edited date
- Continue button

Creating a project opens a short setup flow:

1. Enter the initial story prompt.
2. Select Comic or Manga.
3. Select genre, tone, intended audience, language, and approximate page count.
4. Optionally upload writing, character references, PDFs, or visual references.
5. Create the project and begin outline generation.

The existing owner, admin, editor, and viewer permissions remain unchanged. Viewers may read and comment; editors may revise and generate; admins and owners manage access.

2. Project navigation

Replace Library, Scenes, Shot Design, and Sequence with a guided project stepper:

1. Idea
2. Script
3. Characters & Style
4. Comic
5. Export
6. Anime

The project route resumes at the most recently active stage. Direct stage routes should be available:

- `/studio/frame/projects/:projectId/idea`
- `/studio/frame/projects/:projectId/script`
- `/studio/frame/projects/:projectId/characters`
- `/studio/frame/projects/:projectId/comic`
- `/studio/frame/projects/:projectId/export`
- `/studio/frame/projects/:projectId/anime`

Existing project-level collaborators, comments, save status, history, and realtime updates remain available throughout these screens.

3. Idea and outline generation

The Idea screen contains one prominent prompt field and optional creative controls.

Users can specify:

- Premise
- Genre and subgenre
- Tone
- Audience
- Comic or manga
- Page target
- Language
- Character or world notes

- Required themes
- Content to avoid
- Uploaded references

The first generation produces a structured outline containing:

- Working title
- Logline
- Short synopsis
- Main characters
- Beginning, middle, and ending
- Major story beats
- Proposed page distribution
- Recommended visual direction

The outline remains editable. Regenerating it creates an immutable child version. Users can compare versions, restore an older version, or approve one version for script generation.

No comic images are generated until the user approves the outline.

4. Script generation and editing

The approved outline is converted into a structured comic script.

The script editor presents:

- Story acts or chapters
- Ordered scenes
- Page breakdown
- Panels within each page
- Panel action
- Camera framing
- Dialogue
- Narration
- Sound effects
- Estimated timing for animation

Users can:

- Edit any text directly
- Add, delete, duplicate, split, merge, and reorder scenes or panels
- Regenerate only one scene or panel
- Ask the system to shorten or expand the story
- Lock sections that must not change
- Compare script versions
- Approve a script version

The system must preserve stable character, scene, page, and panel IDs when revising unaffected sections. This allows existing artwork and anime shots to remain attached after small script edits.

Changing approved dialogue marks affected comic pages and audio tracks as outdated without deleting them.

5. Characters and visual style

Before comic generation, users review a generated story bible.

Each character includes:

- Name and role
- Personality
- Appearance
- Clothing and accessories
- Color palette
- Expressions
- Important continuity rules
- Relationships
- Voice assignment
- Uploaded and generated references

Users can generate character sheets containing front, side, expression, and pose references. Character sheets become versioned project assets.

The Style section controls:

- Color or black-and-white
- Line treatment
- Shading
- Detail level
- Background complexity
- Era and atmosphere
- Global palette
- Page borders and gutters
- Speech-balloon style
- Caption and sound-effect typography

An approved character version and approved style version are included in every panel-generation request. Individual panels may override camera, lighting, mood, and composition without changing the global identity.

Provider names and logos are not displayed in the creator interface. The UI displays capability-oriented model names such as "Fast Draft," "Consistent Character," and "High Detail."

6. Comic and manga creation

The Comic screen provides two connected views:

- Page overview
- Focused page editor

Each page contains ordered panels generated from the approved script. The initial layout is selected automatically based on pacing and panel descriptions, but users can switch between tested templates.

Supported page operations include:

- Reorder pages
- Reorder panels
- Change panel layout
- Resize or crop panel art
- Regenerate one panel
- Upload replacement art
- Select a previous art version
- Move or resize speech balloons
- Edit dialogue, captions, narration, and sound effects
- Change lettering style
- Add or remove page numbers
- Mark a page approved

Manga projects use right-to-left page and panel reading order. Comic projects use left-to-right reading order.

7. Professional lettering

Image-generation prompts explicitly request clean artwork without words, logos, signatures, or watermarks.

Writing is rendered separately using editable page elements:

- Speech balloons
- Thought balloons
- Narration boxes
- Sound effects
- Labels
- Page numbers

Every text element stores its content, panel association, position, dimensions, tail position, typography, alignment, and reading order.

The page editor warns about:

- Overflowing text
- Balloons outside safe areas
- Balloons covering important faces
- Unresolved speaker assignments
- Missing fonts
- Empty panels
- Low-resolution artwork

This separation allows users to correct spelling or translate the story without regenerating artwork.

8. Story exports

The Export screen performs a preflight check before allowing publication.

Initial export formats:

- PDF
- CBZ

- ZIP containing numbered page images

PDF options include:

- Cover inclusion
- Page size
- Margins and bleed
- Image quality
- Page numbering
- Metadata
- Reading direction

Exports run as durable background jobs. The backend renders every page from the approved [PageDocument](#), stores the finished artifact in Supabase Storage, and provides a time-limited download URL.

Export history records:

- Source script version
- Character/style versions
- Page versions
- Export settings
- Status
- File size
- Checksum
- Creation time
- Creator

An old export remains downloadable even after the project changes.

One-Click Anime Generation

Anime preflight

The Anime screen uses the approved script, character bible, page layouts, panel artwork, and dialogue.

Before generation, the system checks:

- Every required panel has usable artwork
- The script and comic versions are approved
- Character references are available
- Dialogue speakers are assigned
- Required text, image, video, and voice capabilities are configured
- Estimated generation count and cost
- Expected episode length

The user may select:

- Aspect ratio
- Target duration
- Frame rate

- Voice for each character
- Narration voice
- Motion intensity
- Subtitle style
- Whether to use lip-sync
- Whether to include user-provided music or sound effects

The confirmation screen shows an estimate but does not block generation based on quotas or credits.

Anime production graph

Clicking "Generate anime" creates one parent production run with dependent steps:

1. Convert comic panels into an ordered anime shot plan.
2. Create motion prompts using panel action, camera direction, character references, and continuity context.
3. Generate image-to-video clips for appropriate shots.
4. Use controlled camera movement on still panels when video generation is unnecessary or fails.
5. Generate dialogue and narration audio.
6. Apply lip-sync where configured and appropriate.
7. Place captions using the approved script.
8. Add user-provided music and sound effects where available.
9. Assemble every successful result into a video project.
10. Open the completed draft in the existing video editor.

A failed individual shot does not discard the entire run. The system retries it within configured limits and then uses the approved still panel with a camera move as a visible fallback. The completed run lists all warnings and substitutions.

No dummy media, silent placeholder clips, or fake completion states are permitted.

Anime review

The Anime screen displays shots in story order with:

- Source page and panel
- Generated preview
- Dialogue
- Duration
- Generation status
- Voice status
- Retry and regenerate actions
- Open in video editor action

Regenerating a shot creates a new version. The user chooses which version becomes current.

Video Editor Direction

Story integration

Add "Create anime draft" and "Open in editor" actions that create a `video_project` linked to the source story and anime run.

The generated timeline contains separate tracks for:

- Video and animated panels
- Still-image fallbacks
- Dialogue
- Narration
- Music
- Sound effects
- Captions
- Visual overlays

Timeline items retain their originating page, panel, character, generation job, and asset-version IDs. The editor can therefore offer "Regenerate this shot" or "Open source panel" without losing timing.

Replacing a generated shot preserves the current timeline position, trim, transitions, volume, and captions unless the user explicitly resets them.

TimelineDocument version 2

Replace millisecond-first timing with frame-accurate timing:

- `schemaVersion: 2`
- Rational output frame rate using numerator and denominator
- `startFrame`
- `durationFrames`
- `sourceInFrame`
- Source-media frame rate and timebase
- Stable track and item IDs
- Story and generation lineage
- Transitions, transforms, captions, and audio envelopes
- Real frame-edit results

Provide a deterministic migration adapter from existing `TimelineDocument` version 1. Existing video projects must continue opening without destructive migration.

Professional media preparation

After upload or generation, a worker should:

- Probe duration, codec, dimensions, audio channels, and exact frame rate
- Detect variable-frame-rate media
- Generate browser-friendly proxy media when necessary
- Generate thumbnail sprites
- Generate audio waveforms
- Store source and proxy metadata
- Preserve the original asset

The editor uses proxy media for smooth interaction and original assets for future export.

Editor capabilities to complete

Retain and harden:

- Playback and synchronized playhead
- Smooth pointer scrubbing
- Zoom
- Dragging clips
- Snapping
- Trim handles
- Split at playhead
- Reordering
- Multiple video and audio tracks
- Volume and mute
- Captions
- Transitions
- Undo and redo
- Autosave
- Immutable saved versions
- Unsaved, saving, saved, and conflict states

The current prompt-only frame-edit control must not remain as a metadata-only feature. Selecting a frame should offer real actions:

- Replace exactly one output frame with a generated or uploaded image overlay
- Use the edited frame as a keyframe and regenerate the surrounding shot
- Revert to the original frame or shot version

The resulting generated asset and timeline operation must be visible in playback and retained in saved versions.

Until cloud MP4 export is ready, remove the disabled "Export later" control from the production UI. Keep the existing render backend behind a feature flag for a later milestone.

Data Model Changes

Story tables

Add:

- `story_profiles` - one-to-one project configuration, format, reading direction, language, target length, current stage, and approved-version pointers.
- `story_versions` - immutable outline and script documents with parent lineage, source prompt, model/settings snapshots, approval status, and author.
- `story_characters` - stable character identities.
- `story_character_versions` - immutable descriptions, continuity data, voice settings, and reference asset IDs.
- `story_pages` - stable ordered page identities.
- `story_page_versions` - immutable page-layout and lettering documents.
- `story_panels` - stable ordered panel identities linked to script elements and pages.

- `story_panel_versions` - immutable action, dialogue, prompts, approved artwork, and generation lineage.
- `production_runs` - parent script, comic, export, or anime workflows.
- `production_steps` - dependency-aware units inside a run.
- `story_exports` - source versions, format, settings, status, storage path, checksum, and size.
- `anime_runs` - anime-level settings and source-version pointers.
- `anime_shots` - ordered source panels, timing, prompts, voice assets, video assets, status, and current version.

Extend `generation_jobs` with nullable `production_run_id` and `production_step_id`. Keep the existing Shot Design relationship nullable for compatibility.

Extend comments to support story version, character, page, panel, anime shot, and timeline-item targets.

Extend `video_projects` with optional source story, anime run, and production run IDs.

Important contracts

Define and validate shared Zod/OpenAPI contracts for:

- `StoryDocument`
- `CharacterDocument`
- `PageDocument`
- `LetteringElement`
- `ProductionRun`
- `ProductionStep`
- `AnimePlan`
- `AnimeShot`
- `TimelineDocumentV2`

`StoryDocument` uses stable IDs and contains metadata, characters, scenes, pages, panels, dialogue, narration, sound effects, visual direction, and timing hints.

`PageDocument` uses normalized coordinates for panels and text elements. Text wrapping and final line positions are calculated deterministically so browser previews and exported pages match.

API Changes

All routes continue using `/api/v1`, Zod validation, custom sessions, CSRF protection, permission policies, idempotency keys, and `{ data, error, meta }` envelopes.

Add:

- `GET/PATCH /projects/:id/story-profile`
- `GET /projects/:id/story`
- `POST /projects/:id/story-versions`
- `GET /projects/:id/story-versions`
- `POST /story-versions/:id/approve`
- `POST /story-versions/:id/regenerate-section`
- `GET/POST /projects/:id/characters`
- `POST /characters/:id/versions`

- `POST /character-versions/:id/approve`
- `GET /projects/:id/pages`
- `GET /pages/:id`
- `POST /pages/:id/versions`
- `POST /panels/:id/generations`
- `POST /panel-versions/:id/make-current`
- `POST /production-runs`
- `GET /production-runs/:id`
- `POST /production-runs/:id/cancel`
- `POST /production-steps/:id/retry`
- `POST /projects/:id/story-exports`
- `GET /story-exports/:id`
- `POST /projects/:id/anime-runs`
- `GET /anime-runs/:id`
- `POST /anime-shots/:id/regenerate`
- `POST /anime-runs/:id/create-video-project`

All editable resources carry a revision. Mutations require `expectedRevision`; conflicts return the canonical document for reconciliation.

WebSocket project rooms broadcast production progress, version changes, page approvals, comment events, anime-shot completion, and editor handoff.

Provider and Job Architecture

Change the provider registry from workflow-specific behavior to capability matching:

- `text.structured`
- `text.rewrite`
- `image.generate`
- `image.reference`
- `image.edit`
- `video.image_to_video`
- `voice.tts`
- `video.lip_sync`
- `audio.music`
- `audio.sound_effect`

The UI selects an Origin capability preset. The backend resolves it to an enabled model from the database catalog.

Structured text generation must:

- Request schema-constrained output where supported
- Validate every response
- Attempt bounded repair for invalid responses

- Reject unresolved malformed output
- Store prompt, response, settings, model, and validation failures

Generated provider media must be imported into Supabase Storage immediately after completion. Projects must not depend permanently on expiring provider URLs.

Production runs are dependency graphs processed through pg-boss. Each production step supports queued, running, retrying, succeeded, failed, canceled, and skipped states.

Use bounded concurrency per provider and project so a 24-page comic does not overwhelm provider limits. Idempotency keys prevent duplicated media or cost events during retries.

Missing provider credentials disable the affected capability during preflight instead of failing halfway through the workflow.

Comic Rendering

Create a server-side page renderer driven entirely by [PageDocument](#).

Use:

- Deterministic panel rectangles
- SVG-based balloon and text geometry
- Embedded licensed fonts with Unicode support
- High-resolution artwork from Supabase Storage
- PDF generation with page metadata
- Raster page generation for CBZ and ZIP
- Checksums for completed artifacts

The frontend page editor manipulates normalized coordinates and already-calculated text lines. The backend uses the same positions for export, preventing layout differences between preview and PDF.

Export workers write temporary files outside tracked source directories, upload the completed artifact to Supabase Storage, and remove temporary files after successful import.

Migration and Compatibility

- Add all new tables through forward-only Drizzle migrations.
- Do not delete existing projects, assets, scenes, video projects, or histories.
- Existing projects appear as "Legacy project" and offer an explicit conversion action.
- Conversion creates a story profile and maps ordered scenes into initial story beats or panels.
- Existing Shot Design routes remain temporarily available for old records but disappear from new-project navigation.
- Existing [TimelineDocument](#) version 1 records are upgraded in memory and saved as version 2 only after the first edit.
- Do not rewrite historical timeline versions.
- Keep Origin Sites and Origin Garage unchanged as Coming Soon products.

Implementation Sequence

Phase 1 - Product pivot and contracts

- Add story schemas, database migrations, OpenAPI contracts, feature flags, and timeline-v1 migration.

- Replace new-project creation and project navigation with the guided story workflow.
- Hide the seven Shot Design modes for new projects.
- Preserve existing project access and conversion.

Phase 2 - Prompt, outline, and script

- Add structured text provider capabilities.
- Implement outline generation, editing, version history, comparison, and approval.
- Implement structured script generation and scene/page/panel editing.
- Add section-level regeneration with locked-content preservation.
- Add production-run progress and realtime updates.

Phase 3 - Character consistency and comic generation

- Build character and style-bible editors.
- Generate and approve character-reference sheets.
- Implement panel generation using approved references.
- Build page templates, page editor, version history, lettering, and individual-panel regeneration.
- Add outdated-state tracking after script or style changes.

Phase 4 - Publication export

- Implement deterministic page rendering.
- Add PDF, CBZ, and page-image ZIP jobs.
- Add export preflight, history, Supabase Storage delivery, and signed downloads.
- Verify comic and manga reading directions.

Phase 5 - Anime generation

- Implement anime preflight and cost estimates.
- Generate the shot plan, motion clips, voices, captions, and optional lip-sync.
- Add still-panel motion fallback for failed or intentionally static shots.
- Create a fully populated video project and timeline.
- Add shot review and regeneration.

Phase 6 - Video editor hardening

- Introduce frame-based `TimelineDocument` version 2.
- Add server-side media probing, proxies, thumbnails, and waveforms.
- Complete reliable playhead, drag, trim, split, snapping, multi-track audio, captions, transitions, and autosave.
- Replace metadata-only frame instructions with real frame or shot-generation operations.
- Add story-panel and anime-shot lineage to inspector actions.
- Hide cloud export until its later release milestone.

Phase 7 - Collaboration and production readiness

- Add comments on scripts, characters, pages, panels, shots, and timeline items.
- Add revision-conflict recovery and multi-browser testing.

- Complete responsive, keyboard, screen-reader, reduced-motion, performance, security, and load testing.
- Add monitoring and staged Railway rollout.

Test and Acceptance Plan

Unit tests

Test:

- Story, character, page, anime, and timeline schemas
- Stable-ID preservation
- Story version branching
- Approval and outdated-state rules
- Comic and manga reading order
- Balloon text layout and overflow detection
- Production-step dependency resolution
- Idempotent job and cost recording
- Timeline version-1 migration
- Frame-accurate split, trim, and source-in calculations
- Export checksums

API integration tests

Cover every new endpoint with:

- Successful requests
- Validation failures
- Unauthenticated access
- Viewer/editor/admin permission differences
- Missing resources
- Revision conflicts
- Idempotent replay
- Cancellation
- Partial production failure
- Provider unavailability
- Retry without duplicated outputs or costs

Provider tests

- Use recorded fixtures for every capability adapter.
- Validate structured text repair behavior.
- Verify character-reference payloads.
- Verify provider media is copied into Supabase Storage.
- Run credential-gated sandbox tests for every enabled capability.

End-to-end acceptance scenario

A user must be able to:

1. Create an account and story project.
2. Enter one prompt.
3. Generate, revise, and approve an outline.
4. Generate and edit an 8-page script.
5. Approve characters and a visual style.
6. Generate all comic panels.
7. Regenerate one panel without affecting unrelated pages.
8. Correct dialogue without regenerating artwork.
9. Export a readable PDF and CBZ.
10. Start anime generation.
11. Recover from one deliberately failed shot using the visible still-motion fallback.
12. Open the assembled anime draft in the video editor.
13. Play, scrub, trim, split, reorder, caption, and autosave it.
14. Regenerate one shot and preserve its timeline placement.
15. Reopen the saved project without losing versions or lineage.

Quality gates

- No placeholder or dummy media in production flows.
- No control presented as working unless it performs a real persisted action.
- No dialogue baked into generated artwork.
- No provider URLs used as permanent project storage.
- No destructive migration of existing projects.
- Clean type-check, lint, tests, production builds, security scan, and forbidden-brand scan.
- Keyboard-accessible guided workflow and page editor.
- PDF preview and final export match within defined rendering tolerances.

Rollout and Success Measures

Release behind `STORY_WORKFLOW_ENABLED` and `ANIME_GENERATION_ENABLED` flags.

Track:

- Time from project creation to approved outline
- Percentage reaching approved script
- Percentage generating a complete comic
- Comic export completion rate
- Panel-regeneration rate
- Anime-run completion rate
- Percentage of generated anime drafts opened in the editor
- Production failure and fallback rates

- Repeat project creation
- Generation cost per completed page and per completed minute

Do not enable all users until one complete 8-24-page project can reliably reach PDF export and an editable anime draft without manual database intervention.

Assumptions and Defaults

- Guided approvals are the default; expensive stages never start silently.
- Comic and manga are first-class; vertical webtoon layouts are deferred.
- The initial production target is 8-24 pages and a 1-3-minute anime draft.
- Provider selection remains capability-based and provider-neutral in the UI.
- English is the initial fully QA-tested writing language; storage and fonts are Unicode-ready.
- PDF, CBZ, and page-image ZIP export are included.
- Cloud MP4 export is deferred; the anime result must still be fully playable and editable in-browser.
- Existing roles, authentication, workspaces, Supabase Storage, collaboration, and observational cost tracking remain intact.
- Existing projects and timelines are preserved through additive migrations and compatibility adapters.